

Project Details — MapRover

What Is MapRover?

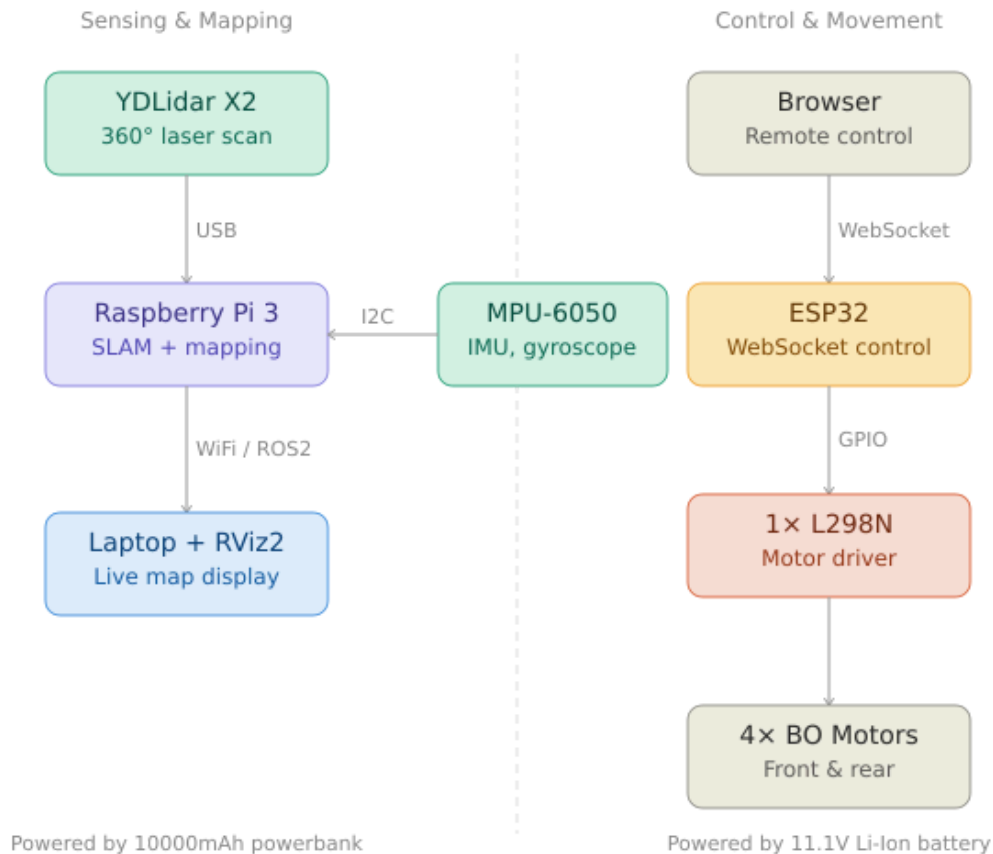
MapRover is a teleoperated rover built on a 6-wheel rocker-bogie chassis, with the front and rear 4 wheels driven, that builds a real-time 2D map of any indoor environment as you drive it around. The operator controls the rover remotely through a browser on their phone or laptop. As the rover moves, a spinning laser sensor (LiDAR) continuously scans the surroundings and the onboard computer assembles a live map — the same principle behind how robot vacuums "see" their rooms, but with far more precision.

The project is inspired by NASA's Mars rovers, specifically the rocker-bogie suspension system, which allows the rover to roll over uneven terrain without tipping. The same mechanism is used here.

The Problem It Solves

Exploring unknown or restricted indoor spaces — collapsed structures, large warehouses, hazardous rooms — is risky for humans. Professional mapping equipment is expensive. MapRover offers an affordable, remotely operated alternative that can navigate a space and generate a usable map in real time.

System Architecture



Movement layer (ESP32): The ESP32 microcontroller hosts its own WiFi access point. The operator connects to it and opens a browser page with directional buttons. Commands are sent over WebSockets to the ESP32, which drives a single L298N motor driver module — OUT1 and OUT2 (Channel A) control the right-side 2 motors (front and rear right), and OUT3 and OUT4 (Channel B) control the left-side 2 motors (front and rear left). The 2 middle wheels on the rocker-bogie arms are passive and not driven.

Mapping layer (Raspberry Pi 3): The Raspberry Pi 3 runs ROS2 Humble on Ubuntu Server 22.04. It reads laser scan data from the YDLidar X2 (connected over USB) and runs the slam_toolbox package to build a 2D map in real time. An MPU-6050 IMU connected to the Pi over I2C feeds rotation data into a robot_localization EKF node, which corrects the map during sharp turns and prevents distortion.

Visualization: The laptop runs RViz2 and connects to the Pi over the same WiFi network using ROS2's built-in multi-machine communication (ROS_DOMAIN_ID). The live map is displayed on the laptop as the rover moves.

Hardware

Component	Purpose	Quantity
ROVER 6WD BO60 Chassis Kit	Base chassis, rocker-bogie suspension, 6x BO60 motors, wheels	1
YDLidar X2	360° laser scanner for mapping	1
Raspberry Pi 3	Mapping computer, runs ROS2 and slam_toolbox	1
ESP32	Motor controller, hosts WiFi and WebSocket server	1
L298N Motor Driver Module	Channel A (OUT1+OUT2) drives right 2 motors, Channel B (OUT3+OUT4) drives left 2 motors	1
MPU-6050 IMU	Gyroscope for rotation correction during mapping	1
11.1V 5000mAh 3S Li-Ion Battery	Powers the motors via L298N	1
10000mAh Powerbank	Powers the Raspberry Pi via USB	1
Aluminium Heatsinks	Prevents Raspberry Pi from throttling	1 set
M3 Nylon Standoffs	Mounts PCBs safely above the metal chassis	As needed
XT60H Connector	Battery connection point	1

Component	Purpose	Quantity
Micro SD Card	OS storage for Raspberry Pi	1
Jumper wires, silicone wire 20AWG	All electrical connections	As needed
Zip ties, double-sided tape	Cable management and component mounting	As needed

The SG90 servo and ultrasonic sensor included in the rover kit are not used in this version of the project. The 2 middle motors on the rocker-bogie arms are also not connected to the driver — only the 4 front and rear motors are driven.

Software Stack

Software	Where it runs	Purpose
Ubuntu Server 22.04 LTS (64-bit)	Raspberry Pi	Operating system required for ROS2 Humble
ROS2 Humble	Raspberry Pi	Robot middleware — connects all software components
slam_toolbox (async)	Raspberry Pi	Builds the 2D map from LiDAR scan data
ydlidar_ros2_driver	Raspberry Pi	Reads data from the YDLidar X2
robot_localization (EKF)	Raspberry Pi	Fuses IMU data for accurate rotation tracking
RViz2	Windows Laptop (via WSL2)	Visualizes the live map
Arduino IDE / ESP32 firmware	ESP32	WebSocket server + motor control logic

Key Design Decisions

Why two separate controllers (ESP32 + Raspberry Pi)? The Raspberry Pi handles computationally heavy work (SLAM, LiDAR processing). Running motor control on the same board would introduce timing conflicts and slow down the mapping. The ESP32 handles motor control independently and reliably at low latency.

Why ESP32 instead of Arduino Nano for motor control? The original plan used the Arduino Nano, but the ESP32 was used because it has built-in WiFi, allowing wireless control without any additional modules. The operator does not need a physical connection to the rover.

Why a powerbank for the Pi instead of the main battery? Motors create large voltage spikes when starting and stopping. Powering the Pi from the same battery as the motors risks corrupting the SD card or crashing the Pi. The powerbank provides clean, stable power completely isolated from the motor circuit.

Why EKF with IMU? Without the IMU, slam_toolbox relied only on LiDAR scan matching for its rotation estimate. During sharp turns, the scan matching briefly fails (the environment looks ambiguous) and the map tears diagonally. The MPU-6050's gyroscope gives an independent, accurate rotation reading that the EKF fuses in to prevent this.